OBJECT-ORIENTED PROGRAMMING WITH C++

ASSIGNMENT 1 – CO1

Design, Implement and Validate a Smart Parking Management and Payment System

SIMATS ENGINEERING

Department of Artificial Intelligence and Data Science

Student Name	APARNA T
Register Number	192524213
Department / Section	B.Tech AI&DS
Academic Year / Semester	2nd Year
Faculty Name	AMIRTHA S
Nature of Assignment	Individual
Duration	1 Week
Total Marks	50
Date of Submission	10 / 11 / 2026
GitHub Repository	https://github.com/aparnathangadurai201537-ux/OOP-IN-C--192524213
Compiler	g++ / C++17

Note: This assignment follows the task pattern of the reference document, but the scenario, examples, code, values and analysis below are independently prepared for a Smart Parking Management System.

MARK SHEET (Rubric-Based Evaluation)

Task	Task Title	Max Marks	Marks Awarded	Remarks
1	Object-Oriented Analysis	4		
2	Tokens, Keywords, Identifiers and Constants	3		
3	Data Types and Compatibility	4		
4	Implicit Conversions	4		
5	Operators and Precedence	4		
6	Control Structures	4		
7	Procedural vs Object-Oriented	4		
8	Input and Output	4		
9	Implementation and Testing	4		
10	Expectation vs Execution	4		
11	Final Recommendation	4		
12A	Impact Analysis	2		
12B	Evaluation of Original Design	2		
12C	Adaptation / Redesign	2		
12D	Evidence-Based Defence	2		
	TOTAL	50		

Scenario – Smart Parking Management and Payment System

A large shopping complex operates a smart parking facility for two-wheelers and four-wheelers. At the entry gate, the system records the vehicle number, vehicle type, parking slot and entry time. At exit, the software calculates the parking fee from the total duration. Different rates are used for two-wheelers and four-wheelers, and an overtime charge is added when a vehicle stays beyond the normal parking limit. Registered customers receive a discount.

The first version of the parking software was developed quickly. Some charges were rounded incorrectly, an exact boundary duration was treated as overtime, invalid vehicle numbers were accepted, and the same parking rates were written in several functions. These issues can lead to incorrect bills and difficult maintenance.

The number of parking transactions may increase from a small parking area to multiple floors and finally to the entire shopping complex. The system therefore needs correct fee calculation, safe input handling, clear object-oriented design and efficient processing.

Design Challenge: The management later plans to introduce EV charging stations, VIP parking spaces and monthly parking membership. Re-evaluate the original design and decide whether it should be retained, modified or redesigned.

TASK 1 – Understand the Problem and Identify the Objects

The parking problem is suitable for object-oriented analysis because several real-world entities have their own data and behaviour.

1. Main Objects

Object / Class	Important Data	Important Behaviour
Vehicle	vehicleNumber, vehicleType	displayVehicle(), validateNumber()
Customer	customerId, name, registeredStatus	applyDiscount(), displayCustomer()
ParkingSlot	slotNumber, slotType, occupied	assignVehicle(), releaseSlot()
ParkingTicket	ticketId, vehicle, entryTime, exitTime	calculateDuration(), calculateFee()

ParkingManager	slots, tickets, rates	allocateSlot(), processExit(), generateBill()

The Vehicle object stores information about the vehicle. ParkingTicket connects one parking session with entry and exit details. ParkingSlot represents the physical parking space. Customer contains registration information used for discount eligibility. ParkingManager coordinates the complete parking operation.

2. OOP Principles

- Encapsulation: parking fee and ticket data can be kept private and changed through controlled methods.
- Abstraction: the user only calls calculateFee() or processExit() without needing to know every internal calculation.
- Inheritance: a base Vehicle class can be extended to TwoWheeler and FourWheeler when vehicle-specific rules grow.
- Polymorphism: a virtual calculateBaseRate() can return a different rate for different vehicle types.

3. Rules for a Correct Parking Session

1. Vehicle number must follow the required format and must not be empty.
2. Vehicle type must be either two-wheeler or four-wheeler.
3. Entry time must be recorded before exit time.
4. Parking duration must be non-negative.
5. Discount must be applied only to eligible registered customers.
6. Final bill must never become negative.
7. Every ticket must refer to one valid parking slot.

Alternative Concept and Complexity Analysis

A procedural design could use structures and separate functions such as createTicket(), calculateDuration(), calculateFee() and releaseSlot(). For one ticket, both procedural and object-oriented approaches take O(1) time for fee calculation because only a fixed number of arithmetic and condition operations are used. Storage for one ticket is also O(1).

The major difference appears during maintenance. In the procedural approach, fee rules may be repeated in several functions. In the object-oriented approach, ParkingTicket and ParkingManager keep related rules together. Therefore, adding a new discount or parking type requires fewer changes in the OO design.

TASK 2 – Tokens, Keywords, Identifiers and Constants

1. Tokens Used in the Parking Program

Token Type	Examples	Use
Keyword	class, private, public, if, else, return	C++ language structure
Identifier	vehicleNumber, parkingFee, ParkingTicket	Names chosen by programmer
Constant	BASE_2WHEELER, BASE_4WHEELER, MEMBER_DISCOUNT	Fixed parking values
Operator	+ , -, *, /, %, >=, &&	Calculation and decision making
Literal	120, 2.5, "TN01AB1234", true	Fixed values written directly
Punctuation	; , { } ( )	Program syntax

2. Naming Examples

Good identifiers: vehicleNumber, entryMinutes, exitMinutes, finalFee, memberDiscount. These names clearly indicate their purpose.

Poor identifiers: x1, aa, temp2, q. Such names make the program difficult to understand and maintain.

3. Recommended Constants

```
const double TWO_WHEELER_RATE = 20.0;
const double FOUR_WHEELER_RATE = 40.0;
const double EXTRA_HOUR_RATE = 15.0;
const double MEMBER_DISCOUNT = 0.10;
const int FREE_LIMIT_MINUTES = 120;
```

Using constants prevents repeated values from appearing in many places. If the management changes the two-wheeler rate, only the corresponding constant needs to be updated.

TASK 3 – Data Types and Compatibility

Data Item	Suggested Type	Reason
Vehicle number	string	Contains letters and digits
Vehicle type	enum class	Only fixed categories are required
Entry time in minutes	int	Whole-minute calculation
Exit time in minutes	int	Whole-minute calculation
Parking duration	int	Duration can be stored as whole minutes
Parking fee	double	Supports rupee values with paise
Discount percentage	double	Fractional percentage
Occupied slot count	int	Whole number of slots
Ticket ID	long long	Safe for growing ticket numbers

Compatibility Example

```
int duration = 155;
double rate = 22.50;
double charge = duration * rate;
std::cout << charge;
```

Here duration is int and rate is double. C++ converts duration to double before multiplication. Therefore the result is a double. This is useful when a whole-number duration is multiplied by a rate containing decimals.

Alternative Type Choice

If all rates were stored as paise using integer values, exact currency arithmetic could be achieved without floating-point rounding. For example, Rs. 22.50 can be represented as 2250 paise. For a beginner-friendly parking bill, double is acceptable, but a production billing system should consider integer paise or a decimal currency type.

TASK 4 – Implicit Conversions

Implicit conversion occurs when C++ automatically changes one data type into another during an expression.

```
int hours = 3;
double hourlyRate = 37.50;
double amount = hours * hourlyRate;


int paid = 200;
double balance = paid - amount;
```

The int value hours is converted to double before multiplication. Similarly, paid is converted to double when it is subtracted from amount.

Parking Example

```
int duration = 150;
int fullHours = duration / 60;       // 2
double remainder = duration % 60;    // 30 converted to double
double fee = fullHours * 40.0 + remainder * 0.50;
```

The division duration / 60 uses integer division, so 150 / 60 gives 2. The remainder is 30. This is important because using integer division without checking the remainder could undercharge the customer.

Alternative: Explicit Conversion

```
double exactHours = static_cast<double>(duration) / 60.0;
std::cout << exactHours;
```

Explicit conversion makes the programmer's intention clearer and is safer when fractional hours are required.

TASK 5 – Operators and Precedence

The parking system uses arithmetic, relational and logical operators.

Expression	Purpose
duration - FREE_LIMIT_MINUTES	Find overtime minutes
overtimeMinutes / 60	Find extra hours
baseFee + extraFee	Calculate subtotal
subtotal * (1 - discount)	Apply membership discount
duration > FREE_LIMIT_MINUTES	Check overtime
vehicleType == FOUR_WHEELER	Select vehicle rate
isMember && validVehicle	Check both conditions
payment >= finalFee	Check whether payment is sufficient

Operator Precedence Example

```
double bill = 40 + 15 * 2;
std::cout << bill;
```

Multiplication has higher precedence than addition. Therefore 15 * 2 is evaluated first and the result is 70.

```
double bill = (40 + 15) * 2;
std::cout << bill;
```

Parentheses force addition first, giving 110. Therefore parentheses should be used whenever a formula may be misunderstood.

Suggested Final Fee Formula

```
subtotal = baseFee + overtimeFee;
finalFee = subtotal * (1.0 - memberDiscount);
```

This separates the calculation into understandable steps and reduces mistakes caused by complex expressions.

TASK 6 – Control Structures

1. Selection

```
if (duration <= 120)
fee = baseFee;
else
fee = baseFee + overtimeFee;
```

if-else is suitable because the system has different actions for normal and overtime parking.

2. Switch

```
switch (vehicleType)
{
case 1:
baseFee = 20;
break;
case 2:
baseFee = 40;
break;
default:
std::cout << "Invalid vehicle type";
}
```

3. Loop

```
for (int i = 1; i <= 5; ++i)
std::cout << "Processing ticket " << i << "\n";
```

A loop is useful when several parking tickets must be processed.

4. While And Break/Continue

```
int attempts = 0;
while (attempts < 3)
{
attempts++;
// read and validate vehicle number
if (validVehicle)
break;
}
```

break can stop repeated input after a valid value is received. continue can skip an invalid record and move to the next iteration.

TASK 7 – Procedural vs Object-Oriented Approach

Point	Procedural Design	Object-Oriented Design
Main unit	Functions and structures	Classes and objects
Fee data	May be shared globally	Stored in controlled objects
Validation	Caller must remember rules	Class methods can enforce rules
Reuse	Function-based reuse	Class and inheritance-based reuse
Change in parking rule	Several functions may need editing	Main fee class can be updated
Large system	Can become difficult to organise	Better separation of responsibilities

Procedural Example

```
double calculateFee(int type, int minutes)
{
    double base = (type == 1) ? 20.0 : 40.0;
    if (minutes > 120)
        base += (minutes - 120) * 0.25;
    return base;
}
```

Object-Oriented Example

```
class ParkingTicket
{
private:
    int duration;
    int vehicleType;


public:
    double calculateFee()
    {
        double base = (vehicleType == 1) ? 20.0 : 40.0;
        if (duration > 120)
            base += (duration - 120) * 0.25;
        return base;
    }
};
```

Both solutions can calculate one ticket in O(1) time. The OO version provides better encapsulation because ticket data and ticket behaviour are kept together.

TASK 8 – Read and Print Data Safely

The program should validate input instead of assuming that every entered value is correct.

```
#include <iostream>
#include <string>
using namespace std;


int main()
{
    string vehicleNumber;
    int duration;


    cout << "Enter vehicle number: ";
    cin >> vehicleNumber;


    cout << "Enter parking duration in minutes: ";
    cin >> duration;


    if (cin.fail())
    {
        cout << "Invalid numeric input";
        return 0;
    }


    if (duration < 0)
    {
        cout << "Duration cannot be negative";
        return 0;
    }


    cout << "Vehicle: " << vehicleNumber << endl;
    cout << "Duration: " << duration << " minutes" << endl;
}
```

Example Input / Output

Input: vehicle number = TN57CD4321, duration = 145 minutes

Output: Vehicle: TN57CD4321; Duration: 145 minutes

Invalid input example: duration = abc → Output: Invalid numeric input.

Output Formatting

```
#include <iomanip>
cout << fixed << setprecision(2);
cout << "Final Fee: Rs. " << finalFee;
```

setprecision(2) displays the bill with two digits after the decimal point, which is suitable for currency presentation.

TASK 9 – Implementation and Testing

Complete Parking Fee Program

```
#include <iostream>
#include <iomanip>
using namespace std;

double calculateFee(int type, int minutes, bool member)
{
    double base = (type == 1) ? 20.0 : 40.0;
    double fee = base;

    if (minutes > 120)
    {
        int extraMinutes = minutes - 120;
        double extraHours = (extraMinutes + 59) / 60;
        fee += extraHours * 15.0;
    }

    if (member)
        fee *= 0.90;

    return fee;
}

int main()
{
    int type, minutes;
    char member;

    cout << "Vehicle type (1=Two Wheeler, 2=Four Wheeler): ";
    cin >> type;
    cout << "Duration in minutes: ";
    cin >> minutes;
    cout << "Registered member? (Y/N): ";
    cin >> member;

    if (type != 1 && type != 2)
    {
        cout << "Invalid vehicle type";
```

```
        return 0;
    }

    if (minutes < 0)
    {
        cout << "Invalid duration";
        return 0;
    }

    bool isMember = (member == 'Y' || member == 'y');
    double fee = calculateFee(type, minutes, isMember);

    cout << fixed << setprecision(2);
    cout << "Final Parking Fee: Rs. " << fee << endl;
    return 0;
}
```

Test Cases

Test	Vehicle	Duration	Member	Expected Fee
1	Two-wheeler	90 min	No	Rs. 20.00
2	Two-wheeler	150 min	No	Rs. 50.00
3	Four-wheeler	125 min	Yes	Rs. 49.50
4	Four-wheeler	240 min	No	Rs. 70.00
5	Invalid type	100 min	No	Invalid vehicle type
6	Two-wheeler	-10 min	No	Invalid duration

Testing covers normal parking, overtime parking, membership discount, larger duration and invalid input. Boundary testing at exactly 120 minutes is especially important because it should not be treated as overtime.

TASK 10 – Expectation vs Execution

Prediction Before Execution

```
#include <iostream>
using namespace std;


int main()
{
    int minutes = 180;
    double base = 40.0;
    double extra = ((minutes - 120 + 59) / 60) * 15.0;
    double fee = base + extra;


    cout << fee << endl;
}
```

Predicted output: 70

Reason: 180 minutes gives 60 overtime minutes. The integer expression rounds this to one additional hour. Therefore 40 + 15 = 55, not 70. The important observation is that the expression actually produces 1 extra hour, so the correct predicted output is 55.

Corrected predicted output: 55

After execution with a C++17 compiler, the program prints 55. The calculation uses integer division after adding 59. For 60 minutes, (60 + 59) / 60 = 1.

Improved Calculation

```
int extraHours = (extraMinutes + 59) / 60;
```

This performs ceiling division for positive whole minutes. It means 1–60 extra minutes are charged as one additional hour. If the business instead wants per-minute billing, this formula should be changed to a per-minute rate.

Boundary Evidence

Duration	Extra Minutes	Charged Extra Hours
120	0	0
121	1	1
179	59	1
180	60	1

181	61	2

This table demonstrates the effect of the selected parking policy. The exact boundary at 120 minutes is treated as free of overtime.

TASK 11 – Recommend and Defend the Final Design

Final Design Recommendation

8. Use classes such as Vehicle, Customer, ParkingSlot, ParkingTicket and ParkingManager.
9. Store fixed rates and discount percentages as named constants rather than repeating numbers.
10. Use string for vehicle numbers, int for whole-minute durations, long long for ticket identifiers and double or integer paise for money.
11. Validate vehicle type, duration and membership status before calculating the bill.
12. Keep fee calculation inside a dedicated class/function so that rule changes are localised.
13. Use clear test cases for normal, overtime, boundary and invalid conditions.

Alternative Concept – Configuration-Based Rates

Instead of compiling rates into the program, rates could be loaded from a configuration file at startup. If there are z rates, reading them takes O(z) time and O(z) storage. The benefit is that administrators can revise rates without recompiling the application.

Approach	**Advantage**	**Limitation**
Hard-coded constants	Simple and fast; no file dependency	Requires recompilation for rate changes
Configuration file	Easy to revise rates	Needs file validation and startup reading
Database	Supports many parking locations and live changes	More complex and needs database handling

For a college prototype, constants are simple and reliable. For a real multi-floor commercial parking system, configuration or database-based rates are more scalable.

TASK 12 – Re-evaluate and Adapt the Design

12A – Impact Analysis

New requirements: EV charging stations, VIP parking and monthly membership.

New Requirement	Impact on Existing System
EV charging	Needs charging slot, charging duration and energy-based fee
VIP parking	Needs a separate slot category and premium pricing
Monthly membership	Needs customer membership record, validity dates and payment status

These changes affect both data and business rules. A simple single-function fee calculator would become difficult to maintain because many special cases would be added.

12B – Evaluation of Original Design

The original design is still useful because Vehicle, ParkingTicket and ParkingManager separate important responsibilities. However, using only one vehicle type variable and one fee function will become less suitable as parking categories increase.

- The encapsulated ticket model should be retained.
- ParkingSlot should support different slot categories.
- Customer should support membership information.
- Fee calculation should be separated into reusable strategies or specialised classes.

12C – Adaptation / Redesign

The recommended redesign introduces a common ParkingVehicle abstraction and separate pricing behaviour.

```
class Vehicle
{
protected:
    string number;
public:
    virtual double baseRate() const = 0;
    virtual ~Vehicle() = default;
};
```

```
class TwoWheeler : public Vehicle
{
public:
    double baseRate() const override { return 20.0; }
};

class FourWheeler : public Vehicle
{
public:
    double baseRate() const override { return 40.0; }
};

class EV : public Vehicle
{
public:
    double baseRate() const override { return 45.0; }
};
```

A ParkingSlot can then contain a slot category such as NORMAL, VIP or EV. A Membership class can store monthly validity and discount rules. This is more flexible than placing every condition into one large if-else chain.

Redesigned Architecture (Text Form)

```
Customer
   |
Membership ----> Discount Policy
   |
ParkingTicket ----> Vehicle
   |                  |-- TwoWheeler
   |                  |-- FourWheeler
   |                  |-- EV
   |
ParkingSlot ----> Normal / VIP / EV
   |
ParkingManager ----> Fee Calculation + Payment + Reports
```

12D – Evidence-Based Defence

The redesign is supported by the following evidence from the earlier analysis:

14. Task 1 showed that each real-world entity has separate data and behaviour, supporting class-based design.
15. Task 3 showed that vehicle numbers, durations, identifiers and currency need different data types.
16. Task 5 showed that fee expressions become clearer when broken into named steps.
17. Task 9 testing showed that boundary and invalid-input cases must be explicitly checked.
18. Task 11 showed that hard-coded rates are easy for a prototype but configuration is better when the system grows.

19. Therefore, the final design should be modified rather than completely discarded.

Final decision: RETAIN the basic object-oriented structure, MODIFY the fee and slot architecture, and EXTEND it with vehicle subclasses, membership handling and flexible slot categories.

Conclusion

The Smart Parking Management and Payment System demonstrates how C++ object-oriented programming can be applied to a practical billing problem. The analysis identifies meaningful objects, selects suitable data types, uses operators and control structures correctly, validates input, and tests normal and boundary cases.

The object-oriented design is preferable because parking data and operations can be grouped into classes. It also provides a clear path for future requirements such as EV charging, VIP parking and monthly membership. Based on the evaluation, the original design should be retained at its core but modified to support the growing business requirements.